SIMATS ENGINEERING

ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.* (BL4)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 30**

Question Count: 5

Code of Conduct

I __G. Ram Charan (192525043)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____ G. Ram Charan _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption,

and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

Task 1: Web & Mobile Clickstream Analytics Platform

1. Distributed Storage, Ingestion, and Batch Processing

- Distributed Storage: The platform likely leverages an object storage data lake such as Amazon S3, Google Cloud Storage (GCS), or an on-premise HDFS (Hadoop Distributed File System) using open table formats (Apache Iceberg, Delta Lake, or Apache Hudi) with columnar storage formats (Apache Parquet or Apache ORC) to achieve high compression ratios and optimize scan speeds.
- Ingestion Framework: High-throughput event gateways like Apache Kafka or AWS Kinesis act as the ingestion backbone, handling millions of incoming clickstream payloads per second via distributed broker partitions. Logstash or Fluentd/Fluent Bit collect and forward edge log streams to Kafka.
- Batch Processing: Distributed compute engines such as Apache Spark (Core/SQL) or AWS EMR run daily and hourly batch jobs to execute heavy aggregations, deduplication, and historical trend computation.

2. Data Separation and Indexing (Structured vs. Semi-Structured)

- Schema Enforcement & Evolution: Incoming JSON payloads (semi-structured click events) are ingested into bronze raw tables. Spark/Delta pipelines parse, validate schemas, and unpack nested JSON keys into structured silver/gold relational tables.
- Indexing & Partitioning: Data is partitioned by ingestion timestamp (year/month/day/hour) and event domain (event_type). Columnar metadata indexing (min/max zone mapping, Bloom filters, and dictionary encoding) eliminates full table scans during analytical querying.

3. Stream-Processing Engines
   - Real-Time Engines: Apache Flink or Spark Structured Streaming consume topics directly from Kafka with sub-second latency, implementing stateful stream operations, watermarking, and event-time session windowing.
   - Use Cases: Live funnel conversion tracking, real-time fraud/bot detection, bounce rate computation, and active visitor counts.
4. Scalability and Partitioning Strategies
   - Kafka Partitioning: Partitioning by hashed user identifiers (user_id or session_id) ensures strict in-order processing per session while distributing load uniformly across broker clusters.
   - Storage & Compute Decoupling: Auto-scaling compute clusters (e.g., Kubernetes-managed Spark executors) scale independently from the underlying cloud object store. Dynamic file sizing and compaction algorithms (such as Delta OPTIMIZE / bin-packing) prevent the "small-file problem."
5. BI and Analytics Layer Connectivity
   - OLAP Engines: Aggregated data from gold tables is loaded or queried via distributed MPP SQL engines like ClickHouse, Apache Pinot, Snowflake, or Trino (Presto).
   - BI Interfaces: Tools like Tableau, Apache Superset, or Power BI connect via JDBC/ODBC connectors to the OLAP engines, utilizing materialized views, caching layers (e.g., Redis), and direct push-down queries for low-latency visual analytics.

Task 2: Real-Time E-Commerce Personalization & Checkout Platform

1. Event Streaming, Profiling Databases, and Recommendation Workflows
   - Event Streaming: Apache Kafka clusters partitioned by user_id capture atomic interaction events (add_to_cart, item_view, checkout_abandoned).
   - Customer Profiling Databases: Ultra-low-latency NoSQL databases like Apache Cassandra, Amazon DynamoDB, or Aerospike maintain dynamic, high-throughput user state, cart state, and historical preferences.
   - Recommendation Pipeline: Candidate generation and collaborative filtering models hosted on Ray Serve, Triton Inference Server, or TensorFlow Serving read incoming real-time context from feature stores (e.g., Feast) and generate personalized product ranks within milliseconds.
2. Data Synchronization (Sessions, Clickstreams, and Transactions)
   - Stream-Table Joins: Stream-processing frameworks (Apache Flink) perform stateful temporal joins between fast-moving clickstream events and slow-moving transaction/inventory updates.
   - Change Data Capture (CDC): Debezium captures transactional database write-ahead logs (WAL) from PostgreSQL/MySQL and streams them directly into Kafka to ensure real-time inventory and checkout state synchronization across analytical caches.
3. Caching, Distributed Query Engines, and Machine Learning Stages
   - Caching Layer: Redis Enterprise or Memcached provides sub-millisecond retrieval of active session attributes, user token caches, and top-N recommendation lists.
   - Distributed Query Engines: Trino (Presto) federates queries across transactional stores (PostgreSQL) and the analytical data lake (Parquet/S3).
   - ML Lifecycle: Offline feature engineering on Apache Spark, offline model training with PyTorch/XGBoost, continuous model evaluation via MLflow, and online sub-10ms feature retrieval via Feast.
4. Merging Structured and Semi-Structured Data

- ELT Pipelines: Structured transactional tabular data (orders, items, billing) and semi-structured interaction logs (nested JSON clickstreams) are normalized using dbt (data build tool) running on Snowflake or Databricks.
- Identity Stitching: Graph engines or deterministic rule sets match anonymous device tokens and session IDs with authenticated customer accounts upon login.

5. Scalability, Fault Tolerance, and Dashboarding
   - Fault Tolerance: Distributed log replication factors (≥ 3) in Kafka, distributed snapshots (Chandy-Lamport state checkpoints) in Flink, and multi-region read replicas in Cassandra prevent data loss.
   - Dashboarding & KPI Monitoring: Microservices push real-time checkout failure metrics and conversion rates to Prometheus/Grafana for site reliability tracking, while executive sales KPIs feed into Power BI/Looker via scheduled aggregation tables.

Task 3: Enterprise Big Data Healthcare & Diagnostics System

1. Hybrid Database Architecture

- **Structured Data:** ACID-compliant, highly available relational systems (e.g., **PostgreSQL**, **CockroachDB**) manage electronic health records (EHR), billing codes, and prescription databases.
- **Unstructured & Time-Series Data:** Diagnostic medical images (DICOM format) and clinical notes are stored in secure, object-based storage (**Amazon S3 with immutable Object Lock**), while high-frequency wearable sensor vitals (ECG, SpO2, heart rate) flow into specialized time-series databases like **TimescaleDB** or **InfluxDB**.

2. Data Integration and Interoperability

- **Standards & Protocols: HL7** (Health Level Seven) and **FHIR** (Fast Healthcare Interoperability Resources) REST APIs serve as standard interchange formats across external hospitals, diagnostic laboratories, and insurance portals.
- **Integration Middleware: Apache NiFi** manages complex data routing, provenance tracking, and edge-to-core ingestion while enforcing schema transformations using **Mirth Connect** (NextGen Connect).

3. Streaming, Disease Prediction, and Clinical ML

- **Streaming Framework: Apache Flink** monitors continuous telemetry feeds against dynamic threshold alerts (e.g., detecting early septic shock or cardiac arrhythmia in ICU beds).
- **Machine Learning:** Computer vision pipelines (Convolutional Neural Networks / Vision Transformers) implemented in **PyTorch** classify anomalies in X-rays/MRIs; tree-based models (XGBoost) calculate patient readmission and sepsis risk scores.

4. Privacy, Encryption, and Compliance (HIPAA, GDPR)

- **Cryptographic Security:** End-to-end data encryption using **AES-256** at rest (with envelope encryption via AWS KMS / HashiCorp Vault) and **TLS 1.3** in transit.
- **Data Anonymization:** De-identification pipelines strip Protected Health Information (PHI) according to HIPAA Safe Harbor guidelines, replacing identifiers with cryptographically salted pseudo-tokens before moving data to analytical zones.
- **Access Governance:** Attribute-Based Access Control (**ABAC**) via **Apache Ranger** and comprehensive audit logging with **Apache Atlas**.

5. Clinical Decision Support (CDS) and Population Health Pipeline

- **CDS Pipeline:** Real-time inference services integrate with EHR systems via FHIR hooks, surfacing real-time drug-interaction warnings and diagnostic risk flags directly into clinicians' interfaces.
- **Population Health:** Long-term cohort analyses, disease spread modeling, and demographic risk studies execute over petabyte-scale data lakes using **Apache Spark** and **Delta Lake**.

Task 4: Smart City IoT, Traffic, and Infrastructure Platform

1. Edge Computing Setup, Distributed Messaging, and Centralized Storage

- **Edge Architecture:** Micro-edge gateways (installed at traffic intersections, smart poles, and transit stations) run lightweight runtimes (**K3s**, **EdgeX Foundry**) to process raw video feeds locally, extract vehicle counts/license plate metadata, and filter noise before upstream transmission.
- **Messaging Tier:** Edge nodes transmit aggregated JSON/Protobuf telemetry via **MQTT** brokers (e.g., **EMQX**, **Eclipse Mosquitto**), which bridge directly into a scalable **Apache Kafka** cluster.
- **Central Storage:** High-scale distributed object storage (**MinIO**, **Amazon S3**) handles long-term historical video clips and system archives, coupled with a lakehouse platform for analytical processing.

2. Geospatial Analytics and Congestion Prediction

- **Spatial Processing: Apache Sedona** (formerly GeoSpark) and **PostGIS** execute distributed spatial joins (e.g., vehicle trajectory intersections, polygon boundary lookups, and routing network analysis).
- **Congestion Forecasting:** Spatio-temporal graph neural networks (ST-GNNs) and LSTM networks run on continuous sensor streams to forecast bottlenecks 30–60 minutes in advance.

3. Time-Series and Location-Aware Databases

- **Time-Series Engine: TimescaleDB** or **InfluxDB** indexes high-frequency sensor readings (air quality, temperature, acoustic sensors) by timestamp and device ID.
- **Location Engine: Elasticsearch** (with Geo-queries) or **PostGIS** manages vehicle location breadcrumbs, geofencing triggers, and dynamic perimeter searches.

4. Lambda/Kappa Architecture for Urban Planning

- **Real-Time Path (Speed Layer):** Real-time sensor events process through **Apache Flink** to dynamically calibrate signal timings and route emergency vehicles.
- **Batch Path (Serving/Batch Layer):** Nightly **Apache Spark** jobs aggregate monthly commuter flows, public transport load factors, and road degradation data into historical cubes for infrastructure planning.

5. Security, Access Control, and Live Visualization

- **Security Framework:** Public Key Infrastructure (PKI) with mTLS ensures secure edge-device authentication. Strict Role-Based Access Control (**RBAC**) isolates transit authorities from public safety agencies.
- **Visualization Layer:** Real-time Operations Centers (IOC) employ **Deck.gl**, **Mapbox GL**, and **Grafana/Kibana** dashboards connected via WebSockets to render 3D digital-twin city maps, live fleet movement, and critical incident alerts.

Task 5: Hybrid Healthcare Analytics, Telemetry & Risk Scoring Platform

1. Hybrid Storage Architecture for Sensitive Medical Data

- **Polyglot Storage Design:** Relational clinical data (encounters, claims, billing) is stored in ACID-compliant databases (**PostgreSQL / Google Cloud Spanner**).
- **Unstructured & Sensor Stores:** Lab reports (PDFs, pathology slide images) are stored in encrypted distributed blob storage (**Azure Blob / S3**), while real-time continuous physiological data (Holter monitors, smartwatches) is channeled into partitioned time-series storage (**TimescaleDB / OpenTSDB**).

2. ETL Pipelines and Interoperability Standards

- **Integration Engines: Apache NiFi** and **Apache Airflow** coordinate distributed data ingestion pipelines, handling source connectivity across disparate clinic networks.
- **Semantic Normalization:** Medical records are standardized to **FHIR v4**, **SNOMED-CT**, and **LOINC** coding taxonomies to guarantee cross-institutional semantic consistency.

3. Real-Time Telemetry and Predictive Health Analytics

- **Event Ingestion & Processing:** High-frequency biometrics stream through **Kafka** into **Apache Flink**, which executes stateful sliding window aggregations to detect acute physiological decline (e.g., sudden arrhythmia or drops in oxygen saturation).
- **Low-Latency Alerts:** Detected anomalies bypass disk queues to trigger instant push notifications and alerts in hospital monitoring stations via low-latency WebSockets.

4. Distributed Processing for Population-Level Analysis

- **Data Processing at Scale: Apache Spark** runs scalable epidemiologic analytics across millions of historical patient records, computing comorbidity patterns, vaccination efficacy, and regional disease spreads.
- **Data Lakehouse:** Open storage layers (**Delta Lake** or **Apache Iceberg**) provide ACID transactional guarantees, time-travel capabilities for audit reproducibility, and high-performance partition pruning for cohort queries.

5. Data Governance, Compliance, and Audit Enforcement

- **Compliance:** Architecture strictly adheres to HIPAA, HITECH, and GDPR mandates.
- **Security Controls:** End-to-end envelope encryption (KMS managed keys), deterministic column-level hashing for patient identifiers, fine-grained access policies managed via **Apache Ranger**, and immutable audit access trails captured via **Apache Atlas**.

6. Machine Learning for Risk Scoring and Clinical Recommendations

- **Model Pipeline:** Gradient Boosting algorithms (**LightGBM**) and Deep Learning models (**Transformers for clinical NLP**) trained on historical features generate real-time patient risk scores (e.g., 30-day readmission risk, mortality indices).
- **Explainability & Serving:** Model outputs incorporate explainability frameworks (**SHAP** / **LIME**) and are served via high-availability microservices (**Triton** / **FastAPI**) into EHR workflows, providing clinicians with actionable, interpretable diagnostic recommendations.